

Hashing I

Shahriar Ivan

Lecturer, Department of CSE, IUT



Why we need hashing

- Think of storing unrelated/unordered data
 - IP addresses and domain names
- Consider conversions between these two forms
- Thus the the idea of hashing was introduced:
 - Reduced $O(\ln(n))$ operations to $O(1)$

Supporting Example

Suppose we have a system which is associated with approximately 150 error conditions where

- Each of which is identified by an 8-bit number from 0 to 255, and
- When an identifier is received, a corresponding error-handling function must be called

We could create an array of 150 function pointers and to then call the appropriate function....

Array of function pointers

```
#include <iostream>

void a() {
    std::cout
        << "Calling 'void a()'"
        << std::endl;
}

void b() {
    std::cout
        << "Calling 'void b()'"
        << std::endl;
}

int main() {
    void (*function_array[150])();
    unsigned char error_id[150];

    function_array[0] = a;
    error_id[0] = 3;
    function_array[1] = b;
    error_id[1] = 8;

    function_array[0]();
    function_array[1]();

    return 0;
}
```

Output:

```
% ./a.out
Calling 'void a()'
Calling 'void b()'
```

What is the problem?

- Unfortunately, this is slow—we would have to do some form of binary search in order to determine which of the 150 slots corresponds to, for example, error-condition identifier id = 198
- If there was a possibility of dynamically adding new error conditions or removing defunct conditions, this would substantially increase the effort required

Alternate way

Create an array of size 256

Assign those entries corresponding to valid error conditions

```
int main() {  
    void (*function_array[256])();  
    for ( int i = 0; i < 256; ++i ) {  
        function_array[i] = nullptr;  
    }  
  
    function_array[3] = a;  
    function_array[8] = b;  
  
    function_array[3]();  
    function_array[8]();  
  
    return 0;  
}
```

Objective

- Our goal:
 - Store data so that all operations are $\Theta(1)$ time
- Requirement:
 - The memory requirement should be $\Theta(n)$

In our supporting example, the corresponding function can be called in $\Theta(1)$ time and the array is less than twice the optimal size

Another Example

Let's try a simpler problem

How do I store your examination grades so that I can access your grades in $\Theta(1)$ time?

Recall that each student is issued an 8-digit number

- How do I store your examination grades so that I can access your grades in $\Theta(1)$ time?
- Suppose Alice has the ID number 20123456
- I can't create an array of size $10^8 \approx 1.5 \times 2^{26}$

Simplification

I could create an array of size 1000

- How could you convert an 8-digit number into a 3-digit number?
- First three digits might cause a problem: almost all students start with 201, 202, 203, 204, or 205
- The last three digits, however, are essentially random

Therefore, I could store Alice's examination grade

```
grade[456] = 86;
```

Simplification

Question:

What is the likelihood that in a class of size 100 that no two students will have the last three digits?

Not very high:

$$1 \cdot \frac{999}{1000} \cdot \frac{998}{1000} \cdot \frac{997}{1000} \cdot \dots \cdot \frac{901}{1000} \approx 0.005959$$

Simplification

Consequently, we have a function that maps a student onto a 3-digit number

We can store something in that location

Storing it, accessing it, and erasing it is $\Theta(1)$

Problem – two or more students may map to the same number:

- Bob has ID 20173456 and scored 85
- Charlie has ID 2024456 and scored 87

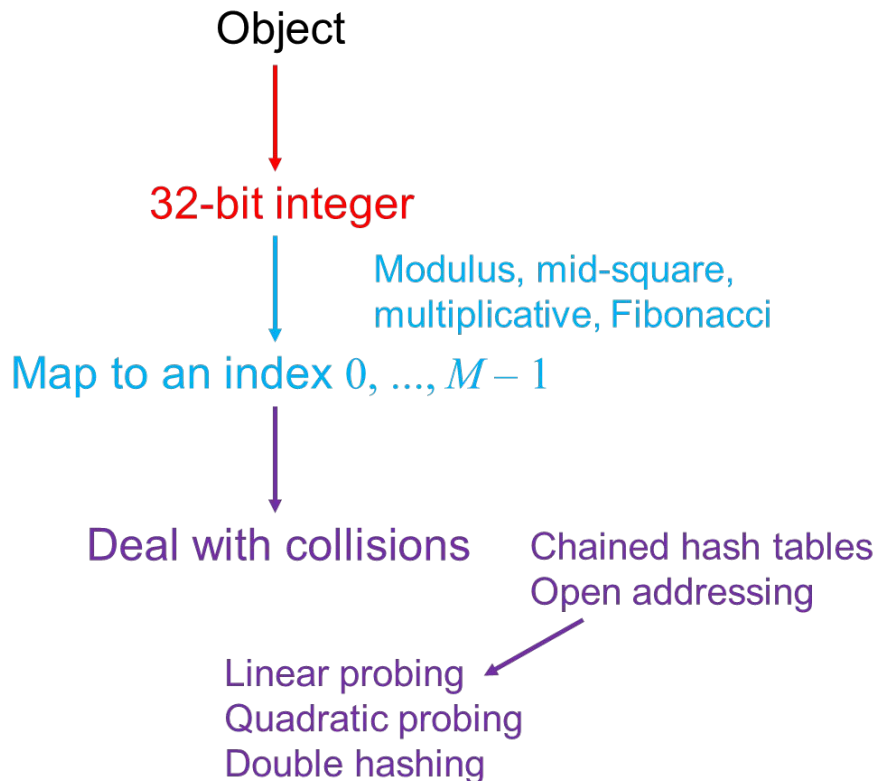
Hashing

- The process of mapping an object or a number onto an integer in a given range is called hashing
- Problem:
 - multiple objects may hash to the same value
 - Such an event is termed a collision
- Hash tables use a hash function together with a mechanism for dealing with collisions

The hash process

Today, we will look at the first problem

- Hashing an object into a 32-bit integer
- Subsequent topics will examine the next steps



Hash function properties

Necessary properties of such a hash function h are:

- Should be fast: ideally $\Theta(1)$
- The hash value must be deterministic
 - It must always return the same 32-bit integer each time
- Equal objects hash to equal values
 - $x = y \Rightarrow h(x) = h(y)$
- If two objects are randomly chosen, there should be only a one-in- 2^{32} chance that they have the same hash value

Types of hash functions

We will look at two classes of hash functions

- Predetermined hash functions (explicit)
- Arithmetic hash functions (implicit)

Predetermined hash functions

The easiest solution is to give each object a unique number

```
class Class_name {
private:
    unsigned int hash_value; // int:           $-2^{31}, \dots, 2^{31} - 1$ 
                             // unsigned int:   $0, \dots, 2^{32} - 1$ 

public:
    Class_name();
    unsigned int hash() const;
};

Class_name::Class_name() {
    hash_value = ???;
}

unsigned int Class_name::hash() const {
    return hash_value;
}
```

Predetermined hash functions

For example, an auto-incremented static member variable

```
class Class_name {  
    private:  
        unsigned int hash_value;  
        static unsigned int hash_count;  
    public:  
        Class_name();  
        unsigned int hash() const;  
};  
  
unsigned int Class_name::hash_count = 0;
```

```
Class_name::Class_name() {  
    hash_value = hash_count;  
    ++hash_count;  
}  
  
unsigned int Class_name::hash() const {  
    return hash_value;  
}
```

Predetermined hash functions

Predetermined hash values give each object a unique hash value

This is not always appropriate:

- Objects which are conceptually equal:
 - Rational x(1, 2);
 - Rational y(3, 6);
- Strings with the same characters:
 - string str1 = "Hello world!";
 - string str2 = "Hello world!";
- These hash values must depend on the member variables
- Usually this uses arithmetic functions

Arithmetic Hash Values

An arithmetic hash value is a deterministic function that is calculated from the relevant member variables of an object

We will look at arithmetic hash functions for:

- Rational numbers, and
- Strings

Rational number class

What if we just add the numerator and denominator?

```
class Rational {  
    private:  
        int numer, denom;  
    public:  
        Rational( int, int );  
};  
  
unsigned int Rational::hash() const {  
    return static_cast<unsigned int>( numer ) +  
        static_cast<unsigned int>( denom );  
}
```

Rational number class

We could improve on this: multiply the denominator by a large prime:

```
class Rational {  
    private:  
        int numer, denom;  
    public:  
        Rational( int, int );  
};  
  
unsigned int Rational::hash() const {  
    return static_cast<unsigned int>( numer ) +  
        429496751*static_cast<unsigned int>( denom );  
}
```

Rational number class

For example, the output of

```
int main() {  
    cout << Rational( 0, 1 ).hash() << endl;  
    cout << Rational( 1, 2 ).hash() << endl;  
    cout << Rational( 2, 3 ).hash() << endl;  
    cout << Rational( 99, 100 ).hash() << endl;  
  
    return 0;  
}
```

Is:

```
429496751  
858993503  
1288490255  
2239
```

Note that arithmetic operations wrap on overflow

Rational number class

This hash function does not generate unique values

The following pairs have the same hash values:

0/1	1327433019/800977868
1/2	534326814/1480277007
2/3	820039962/1486995867

Finding rational numbers with matching hash values is very difficult:

- Finding these required the generation of 1,500,000,000 random rational numbers

It is fast: $\Theta(1)$

It does produce an even distribution

Rational number class

Problem:

The rational numbers $1/2$ and $2/4$ have different values

The output of

```
int main() {  
    cout << Rational( 1, 2 ).hash();  
    cout << Rational( 2, 4 ).hash();  
    return 0;  
}
```

is:

```
858993503  
1717987006
```

Rational number class

Solution: divide through by the greatest common divisor

```
Rational::Rational( int a, int b ):numer(a), denom(b) {  
    int divisor = gcd( numer, denom );  
    numer /= divisor;  
    denom /= divisor;  
}  
  
    int gcd( int a, int b) {  
        while( true ) {  
            if ( a == 0 ) {  
                return (b >= 0) ? b : -b;  
            }  
  
            b %= a;  
  
            if ( b == 0 ) {  
                return (a >= 0) ? a : -a;  
            }  
            a %= b;  
        }  
    }  
}
```

Rational number class

Problem:

The rational numbers $1/2$ and $-1/-2$ have different values

The output of

```
int main() {  
    cout << Rational( 1, 2 ).hash();  
    cout << Rational( -1, -2 ).hash();  
    return 0;  
}
```

is:

858993503

3435973793

Rational number class

Solution: define a normal form

Require that the denominator is positive

```
Rational::Rational( int a, int b ):numer(a), denom(b) {  
    int divisor = gcd( numer, denom );  
    divisor = (denom >= 0) ? divisor : -divisor;  
    numer /= divisor;  
    denom /= divisor;  
}
```

String class

Two strings are equal if all the characters are equal and in the identical order

A string is simply an array of bytes:

Each byte stores a value from 0 to 255

Any hash function must be a function of these bytes

String class

We could, for example, just add the characters:

```
unsigned int hash( const string &str ) {  
    unsigned int hash_value = 0;  
  
    for ( int k = 0; k < str.length(); ++k ) {  
        hash_value += str[k];  
    }  
  
    return hash_value;  
}
```

String class

Not very good:

- Slow run time: $\Theta(n)$
- Words with the same characters hash to the same code:
 - "form" and "from"
- A poor distribution

String class

Let the individual characters represent the coefficients of a polynomial in x :

$$p(x) = c_0 x^{n-1} + c_1 x^{n-2} + \cdots + c_{n-3} x^2 + c_{n-2} x + c_{n-1}$$

Use Horner's rule to evaluate this polynomial at a prime number, e.g., $x = 12347$:

```
unsigned int hash( string const &str ) {  
    unsigned int hash_value = 0;  
  
    for ( int k = 0; k < str.length(); ++k ) {  
        hash_value = 12347*hash_value + str[k];  
    }  
  
    return hash_value;  
}
```

String class

Problem: Horner's rule runs in $\Theta(n)$

```
"A Elbereth Gilthoniel,\n  Silivren penna miriel\n  O menal aglar elenath!\n  Na-chaered palan-diriel\n  O galadhremmin ennorath,\n  Fanuilos, le linnathon\n  nef aear, si nef aearon!"
```

J.R.R. Tolkien

String class

Use characters in locations $2^k - 1$ for $k = 0, 1, 2, \dots$:

```
"A_Elbereth Giltthoniel,\nSilivren_penna miriel\nO menal aglar elenath!\nNa-chaered palan-diriel\nO galadhremmin ennorath,\nFanuilos, le linnathon\nnef aear, si nef aearon!"
```

String class

The run time is now $\Theta(\ln(n))$:

```
unsigned int hash( const string &str ) {  
    unsigned int hash_value = 0;  
  
    for ( int k = 1; k <= str.length(); k *= 2 ) {  
        hash_value = 12347*hash_value + str[k - 1];  
    }  
  
    return hash_value;  
}
```

End of Lecture